INSTITUTO SUPERIOR SANTO DOMINGO

Laboratorio de Programación 3

Guía Didáctica 08 — Upload de archivos: del control FileUpload al IFormFile (VS 2022 / 2026 · .NET 8)

Guía Didáctica 08 Duración 2 horas reloj Profesor Christian Mansilla

Carrera Tecnicatura Superior en Desarrollo de Software

🎯 Objetivos de la clase

- **Subir un archivo desde el navegador y guardarlo en el servidor**: el campo de archivos del formulario, la propiedad `IFormFile` del PageModel y el método que copia el contenido al disco.
- **Guardar el archivo en una carpeta propia del sitio** (`wwwroot/imagenes`) en lugar de la raíz, y saber **dónde vive el sitio** en el disco: `IWebHostEnvironment.WebRootPath` , el heredero de `Server.MapPath(".")` .
- **Leer las propiedades del archivo subido**: el **nombre** (`FileName`), el **tamaño en bytes** (`Length`) y el **tipo** (`ContentType`), y usar el tamaño para limitar el peso.
- **Validar que no exista otro archivo con el mismo nombre** antes de guardarlo, con `System.IO.File.Exists` — el mismo método que usaba el apunte.
- **Validar el tipo y el peso del archivo**: los atributos del modelo, los límites de la petición y el mensaje que ve el usuario.
- **Combinar el upload con el acceso a datos**: guardar en la tabla el *nombre* del archivo (no el archivo) y mostrar la foto con una etiqueta `<img>` servida desde la carpeta de estáticos.

💡 Introducción

El tema de esta clase es una actividad muy común en un sitio web: **el envío de un archivo desde el cliente y su almacenamiento en el servidor**. El apunte lo presenta con el control `FileUpload` y su método `SaveAs` , y después lo lleva al terreno de siempre: guardar el archivo en una carpeta propia, mostrar sus propiedades (tamaño, nombre y tipo), no pisar un archivo que ya existe, y usar todo eso en un ejercicio con base de datos donde la foto de cada auto se guarda en el disco y **su nombre** queda en la tabla.

Ese recorrido se traduce casi entero en tres ideas. La primera: en Razor Pages **no hay un control de archivos que arrastrar**; el campo se escribe con una etiqueta HTML (`<input type="file">`), el archivo viaja en el formulario y el servidor lo recibe en una propiedad de tipo `IFormFile` . La segunda: el

`SaveAs` del apunte se convierte en **copiar el contenido del archivo a un** `FileStream`, y el `Server.MapPath(".")` que armaba la ruta se convierte en `IWebHostEnvironment.WebRootPath`, inyectado por constructor. La tercera: las propiedades que el apunte leía en `FileUpload1.PostedFile` — `ContentLength`, `FileName` y `ContentType` — son las propiedades del propio `IFormFile`: `Length`, `FileName` y `ContentType`. El objeto intermedio desaparece porque ya no es necesario.

Hay un punto que no cambia en nada, y conviene decirlo: **la validación del nombre duplicado usa exactamente el mismo método**. El apunte importa `System.IO` para poder llamar a `File.Exists`; acá ese *using* ya viene puesto por la plantilla del proyecto, así que la línea se escribe igual: `System.IO.File.Exists(ruta)`. Es el único punto de la clase donde el código viejo y el nuevo son literalmente el mismo.

Y cierra la parte de base de datos con la misma continuidad de la guía 06: el `SqlDataSource` del apunte —con sus `InsertParameters`, su `SelectQuery` y su `DeleteQuery` — es el contexto de EF Core y sus consultas LINQ. La foto se guarda como archivo y en la tabla queda el **nombre**: esa división entre *archivo en el disco* y *dato en la tabla* la hace igual el apunte y la hace igual este documento.

📖 Actualización: de VS 2008 a VS 2022 / 2026

Esta es la traducción directa, control por control y método por método, de lo que el apunte configura en el diseñador y escribe en el code-behind. A la izquierda, la pieza de Web Forms; a la derecha, dónde vive eso mismo en un proyecto Razor Pages.

Web Forms (VS 2008)	Razor Pages / .NET 8	Qué hace
El control `FileUpload` (arrastrado al WebForm)	`<input asp-for="Archivo" type="file" />` + `[BindProperty] public IFormFile? Archivo`	Elegir y enviar un archivo
El `<form runat="server">` (el WebForm pone el `enctype` solo)	`<form method="post" enctype="multipart/form-data">`	Que el archivo viaje en el envío
`FileUpload1.SaveAs(ruta)`	`Archivo.CopyToAsync(new FileStream(ruta, FileMode.Create))`	Grabar el archivo subido en el disco
`Server.MapPath(".")`	`A WebHostEnvironment.WebRootPath` (inyectado por constructor)	Dónde vive el sitio en el disco
`Server.MapPath(".") + "/imagenes/" + FileName`	`Path.Combine(env.WebRootPath, "imagenes", Archivo.FileName)`	Armar la ruta del subdirectorio
La carpeta se crea desde el *Explorador de soluciones*	Clic derecho sobre `wwwroot` → Agregar → Nueva carpeta	Crear la carpeta de destino
`FileUpload1.FileName`	`Archivo.FileName`	El nombre del archivo
`FileUpload1.PostedFile.ContentLength`	`Archivo.Length`	El tamaño en bytes
`FileUpload1.PostedFile.ContentType`	`Archivo.ContentType`	El tipo del archivo
`FileUpload1.PostedFile.FileName`	`Archivo.FileName`	El nombre del archivo (el mismo dato)
`System.IO.File.Exists(ruta)`	`System.IO.File.Exists(ruta)`	Saber si ese archivo ya existe
`System.IO.File.Delete(ruta)`	`System.IO.File.Delete(ruta)`	Borrar el archivo del disco
Límite de peso revisado a mano con `ContentLength`	`[RequestSizeLimit(bytes)]` y `[RequestFormLimits(...)]` + el chequeo de `Archivo.Length`	Limitar el peso del archivo
(no existía: se validaba a mano el nombre)	`[FileExtensions(Extensions = "jpg,jpeg,png")]`	Limitar el tipo de archivo
`Page_Load`	`OnGet`	Al entrar a la página
`Button1_Click(object sender, EventArgs e)`	`OnPostAsync` (con `await`) / `OnPost`	Al presionar el botón
`Label1.Text = "Archivo subido"`	Una propiedad del PageModel (`public string? Mensaje`) + `@Model.Mensaje`	Mostrar el resultado

Web Forms (VS 2008)	Razor Pages / .NET 8	Qué hace
`Image1.ImageUrl = registro["foto"]`	`<img src="/imagenes/@Model.Auto.Foto" />`	Mostrar la foto guardada
`DropDownList` + "Elegir origen de datos"	`<select asp-for="Codigomarca" asp-items="Model.MarcasSelect">`	La lista de marcas
`SqlDataSource` + `ConnectionString`	El `AbmContext` inyectado en el constructor (EF Core, guía 04)	Conectarse y consultar la base
`InsertQuery` + `InsertParameters["x"].DefaultValue = TextBox1.Text` + `Insert()`	Un objeto nuevo + `Add` + `SaveChanges()`	Dar de alta una fila
`SelectQuery` + `SqlDataReader` + `Read()`	La consulta LINQ (`Find` , `Where` , `ToList`)	Traer las filas
`DeleteQuery` + `Delete()` + `File.Delete(...)`	`Find` → `Remove` + `SaveChanges()` + `System.IO.File.Delete(...)`	Borrar la fila y su foto
`GridView` con "Elegir origen de datos"	`<table>` + `@foreach` (guía 06)	Dibujar los datos en pantalla
Armar las etiquetas `<img>` como texto en una `Label`	`@foreach` sobre la lista de fotos, con una `<img>` por vuelta	Mostrar varias fotos
`Response.Redirect("Default2.aspx")`	`RedirectToPage("/Index")`	Ir a otra página

Sobre los operadores de las condiciones: en el código de esta guía las comparaciones se escriben con `<=` , `>=` y `!=` (y los chequeos de nulo con `is null` / `is not null`). La lógica es la misma que con el operador de igualdad; el motivo es práctico (el editor del profesor convierte el signo de igualdad al copiar el código y la comparación se rompe). La **igualdad** —que es la comparación más frecuente en las consultas— se escribe con el método `.Equals(...)` : `a.Codigomarca.Equals(Codigomarca)` genera el mismo `where codigomarca = @codigomarca` del apunte.

📖 Temas teóricos

El control FileUpload y la propiedad PostedFile

El apunte describe el `FileUpload` como la componente que **encapsula el envío y la recepción de un archivo** en el servidor web, y aclara que se encuentra en la pestaña de componentes "Estándar" del Cuadro de herramientas. En Razor Pages no hay control que arrastrar: el campo se escribe en la vista con la etiqueta HTML de archivos y el *tag helper* la enlaza a una propiedad del PageModel:

```razor
<form method="post" enctype="multipart/form-data">
    <label asp-for="Archivo">Archivo:</label>
    <input asp-for="Archivo" type="file" />
    <button type="submit">Confirmar</button>
</form>
```

```razor
    // El FileUpload del apunte, ahora como propiedad del formulario:
    // el archivo que llega en el envío se guarda acá
    [BindProperty]
    public IFormFile? Archivo { get; set; }
```

Dos cosas de la etiqueta `<form>` que el apunte no tenía que escribir y ahora sí: `method="post"` y `enctype="multipart/form-data"`. El `enctype` es el formato con el que viaja el archivo; sin él, el formulario se envía pero **el archivo no llega y la propiedad queda nula**. Es el error número uno de esta clase. El `type="file"` se escribe explícitamente porque el *tag helper* no lo deduce del tipo `IFormFile`.

La propiedad `PostedFile`, que en el apunte era la "puerta" a los datos del archivo subido, no tiene equivalente porque **sus propiedades están en el propio `IFormFile`**. Los nombres cambian en un solo caso (`ContentLength` pasa a llamarse `Length`) y el significado es idéntico:

En el apunte (Web Forms)	En Razor Pages	Qué devuelve
`FileUpload1.FileName`	`Archivo.FileName`	El nombre del archivo, tal como lo veía el cliente
`FileUpload1.PostedFile.ContentLength`	`Archivo.Length`	El tamaño en bytes
`FileUpload1.PostedFile.ContentType`	`Archivo.ContentType`	El tipo de archivo (`image/jpeg`, `application/pdf`...)
`FileUpload1.PostedFile.FileName`	`Archivo.FileName`	El mismo nombre del archivo
`FileUpload1.PostedFile.SaveAs(ruta)`	`Archivo.CopyToAsync(stream)`	Grabar el contenido subido
(no existía: se leían los bytes y se grababan a mano)	`Archivo.OpenReadStream()`	El contenido como `Stream`, para leerlo o procesarlo

El `IFormFile` es el archivo, no su contenido. No hay que confundirlo con un arreglo de bytes: representa el archivo que vino en la petición y da tres cosas —sus **datos** (`FileName`, `Length`, `ContentType`), su **contenido** (`OpenReadStream()` y `CopyToAsync(...)`) y la **posibilidad de copiarlo** al destino que se le indique. El apunte usaba el control y el objeto `PostedFile` para lo mismo; acá es un solo objeto.

`Server.MapPath(".")` → `WebRootPath` y la carpeta del sitio

El apunte obtiene el camino donde se almacena la página ASPX actual con `Server.MapPath(".")`. En un proyecto Razor Pages el sitio web vive en una carpeta llamada `wwwroot` —la única que se sirve al

navegador— y su ruta absoluta en el disco la da la propiedad `WebRootPath` de `IWebHostEnvironment`, que se pide por constructor igual que el contexto de la base:

En el apunte (Web Forms)	En Razor Pages
`Server.MapPath(".")`	`env.WebRootPath` — la carpeta `wwwroot` del proyecto
`Server.MapPath(".") + "/imagenes/" + FileName`	`Path.Combine(env.WebRootPath, "imagenes", Archivo.FileName)`
Crear la carpeta desde el *Explorador de soluciones* (botón derecho → Nueva carpeta)	Clic derecho sobre `wwwroot` en el *Explorador de soluciones* → Agregar → **Nueva carpeta**, con el nombre `imagenes`
El `+ "/imagenes/"` concatenado a mano	`Path.Combine(...)`: arma la ruta con el separador correcto del sistema

Dos consecuencias prácticas. La primera: **el archivo guardado adentro de `wwwroot` se ve desde el navegador sin configurar nada**, con una dirección del tipo `/imagenes/foto.jpg`; si se guardara fuera de `wwwroot` habría que agregar un *middleware* de archivos estáticos para poder mostrarlo. La segunda: como `WebRootPath` es una ruta absoluta, para guardar datos que **no** deben ser públicos (un archivo de texto con datos, por ejemplo) se usa `ContentRootPath`, que es la raíz del proyecto — el tema de la guía 02.

El nombre del archivo nunca se usa tal cual para armar una ruta. El nombre lo escribe el usuario y podría venir con una ruta adentro (`..\..\otro.txt`). La defensa es de una línea: `Path.GetFileName(Archivo.FileName)`, que se queda **solo con el nombre** y descarta cualquier carpeta. En los ejemplos de esta guía se usa siempre, aunque el apunte no lo hiciera.

SaveAs → copiar el contenido a un `FileStream`

El apunte graba el archivo con una sola línea: `this.FileUpload1.SaveAs(ruta)`. En Razor Pages el archivo se recibe como `IFormFile` y se copia al destino con `CopyToAsync`, en un bloque `using` que abre el archivo de destino y lo cierra al salir (aunque haya una excepción):

```
// El SaveAs del apunte: se arma la ruta (WebRootPath + "imagenes" + el
// nombre del archivo) y se copia el contenido subido al disco
string ruta = Path.Combine(_env.WebRootPath, "imagenes", Path.GetFileName(Archivo.FileName));

using (FileStream destino = new FileStream(ruta, FileMode.Create))
{
    await Archivo.CopyToAsync(destino);
}
```

Tres detalles del bloque:

- `FileMode.Create` crea el archivo o lo sobrescribe si ya existe: es exactamente el comportamiento que el apunte describe ("esto conlleva a que se pise el archivo antiguo por el nuevo"). El `File.Exists` del punto siguiente es el que evita que eso pase.

- `await` obliga a que el manejador sea asíncrono: `public async Task<IActionResult> OnPostAsync()`. Es el `Button1_Click` del apunte, con la firma que pide la operación de disco.
- **El bloque** `using` es el que garantiza el cierre del `FileStream`. Si se abriera el archivo sin cerrarlo, el archivo quedaría bloqueado en Windows.

El apunte, en su punto 2, recomienda **no mezclar los archivos que suben los usuarios con las páginas del sitio**: por eso la carpeta `imagenes`. En Razor Pages eso se cumple igual (se guarda en `wwwroot/imagenes`, no suelto en `wwwroot`), y con una ventaja: al estar adentro de `wwwroot` la carpeta se sirve al navegador, así que la foto se muestra con una etiqueta `<img>` común.

Las propiedades del archivo: `Length`, `FileName` y `ContentType`

El apunte hace un tercer webform para ver cómo se accede a las propiedades del `FileUpload` y muestra tres de ellas en tres `Label`. En Razor Pages las tres son propiedades del `IFormFile` y se muestran igual, con propiedades del PageModel y `@Model` en la vista:

Dato	En el apunte	En Razor Pages	Ejemplo
El tamaño en bytes	`FileUpload1.PostedFile.ContentLength.ToString()`	`Archivo.Length` (es un `long`)	`184233`
El nombre del archivo	`FileUpload1.FileName`	`Archivo.FileName`	`foto-alfa.jpg`
El tipo del archivo	`FileUpload1.PostedFile.ContentType`	`Archivo.ContentType`	`image/jpeg`

El apunte remarca para qué sirve el tamaño: **"el tamaño del archivo nos puede ser útil si queremos limitar el peso del mismo"**. Eso se ve en el punto siguiente de esta sección.

Validar el nombre duplicado con `File.Exists`

El apunte lo plantea claro: puede suceder que en la carpeta donde se almacena el archivo **exista otro con el mismo nombre**, y entonces el nuevo pisa al viejo. El método estático `Exists` devuelve verdadero si el archivo ya existe; si no existe, se graba. En Razor Pages es **el mismo método, con el mismo nombre**:

```
// El File.Exists del apunte: si el archivo ya está, se avisa y no se graba
string ruta = Path.Combine(_env.WebRootPath, Path.GetFileName(Archivo.FileName));

if (System.IO.File.Exists(ruta))
{
    Mensaje = "Existe un archivo con dicho nombre en el servidor";
}
else
{
    using (FileStream destino = new FileStream(ruta, FileMode.Create))
    {
        await Archivo.CopyToAsync(destino);
    }

    Mensaje = "Archivo subido";
}
```

El apunte aclara que "para utilizar la clase `File` debemos importar el espacio de nombres `System.IO`". Acá ese *using* ya viene incluido por la plantilla del SDK web, así que se puede escribir directamente `File.Exists(ruta)`; en los ejemplos se escribe completo (`System.IO.File.Exists`) solo para que se vea que es la misma clase del apunte. Nótese que en el camino del `else` se graba **solo si no existía**: es el mismo `if / else` del apunte, con el mensaje en una propiedad del PageModel en lugar de en `Label1`.

Validar el tipo y el peso del archivo

El apunte solo mira el `ContentLength` a mano. En una app publicada hay **tres niveles**, y conviene tenerlos claros porque actúan en momentos distintos. El primero es **el tipo del archivo**, con un atributo de DataAnnotations sobre la propiedad; el segundo es **el peso**, que el mensaje al usuario lo da el código, pero el servidor puede rechazar antes con un límite; el tercero es **el tope de la petición**, que es el que se configura para que una subida enorme no llegue siquiera a ejecutar el manejador:

Qué se quiere	Cómo se escribe	Qué pasa si no cumple
Que el archivo sea de cierto tipo	`[FileExtensions(Extensions = "jpg,jpeg,png", ErrorMessage = "...")]` sobre la propiedad	`ModelState.IsValid` da falso y se muestra el mensaje
Limitar el peso (mensaje al usuario)	En el manejador: `if (Archivo.Length >= LimiteBytes) { ModelState.AddModelError(...); }`	El usuario ve el mensaje y vuelve a elegir el archivo
Limitar el peso (que el servidor ni lo reciba)	`[RequestSizeLimit(2097152)]` en la clase del PageModel, y `[RequestFormLimits(MultipartBodyLengthLimit = 2097152)]`	La petición se rechaza antes de llegar al manejador

```
using System.ComponentModel.DataAnnotations;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

    // El tope de la petición: 2 MB. Son atributos de la PÁGINA (van sobre la
    // clase del PageModel), no del archivo: actúan antes de que corra el código.
    [RequestSizeLimit(2097152)]
    [RequestFormLimits(MultipartBodyLengthLimit = 2097152)]
    public class subirvalidandoModel : PageModel
    {
        private const long LimiteBytes = 2097152;   // 2 MB

        // El tipo permitido: el apunte no lo validaba, el atributo sí
        [FileExtensions(Extensions = "jpg,jpeg,png",
            ErrorMessage = "Solo se aceptan archivos .jpg, .jpeg o .png")]
        [BindProperty]
        public IFormFile? Archivo { get; set; }
    }
```

Los dos límites por defecto del servidor. Para que estos atributos hagan falta hay que saber que ya existen dos topes: Kestrel rechaza las peticiones de más de **30.000.000 bytes** (unos 28,6 MB) y el

formulario multipart tiene un tope propio de **134.217.728 bytes (128 MB)**. Si el límite que se quiere es *más chico* que esos, se pone con los atributos de la página; si fuera *más grande*, habría que subir los dos.

El archivo y la base: guardar el nombre, mostrar la foto

La última pieza es la que une esta clase con las anteriores. El apunte guarda en la tabla `autos` la columna `foto varchar(100)`, que **no contiene la imagen sino su nombre**; el archivo, en cambio, queda en el disco. En Razor Pages es igual, y el acceso a la base es el de la guía 06: la entidad con sus atributos `[Table]`, `[Key]` y `[Column]`, el `DbSet` en el contexto, y las operaciones con `Add` / `Remove` / `SaveChanges()`. La foto se muestra con una etiqueta común que apunta a la carpeta de estáticos:

```
<!-- La foto está en wwwroot/imagenes y en la tabla está su nombre -->
<img src="/imagenes/@Model.Auto.Foto" alt="Foto del auto @Model.Auto.Patente" width="300" />
```

Cuando hay **varias** fotos —el punto (d) del ejercicio resuelto— el apunte arma las etiquetas `<img>` como si fueran texto dentro de una `Label`, porque un control `Image` muestra una sola imagen. Acá no hace falta armar HTML a mano: la lista de fotos va al PageModel y la vista la recorre con un `@foreach`, una `<img>` por vuelta. Es más corto y, además, **Razor escapa el valor** de cada dato, así que no hay forma de que un nombre de archivo inyecte HTML.

Errores frecuentes (y cómo se solucionan)

- **La propiedad del archivo siempre llega nula.** Falta `enctype="multipart/form-data"` en el `<form>`, o el nombre del campo no coincide con el de la propiedad. Sin ese `enctype`, el formulario se envía sin el archivo.
- `UnauthorizedAccessException` **al guardar.** La carpeta de destino no existe, o la ruta armada **no incluye el nombre del archivo**. Hay que crear `wwwroot/imagenes` y armar la ruta con `Path.Combine(..., Path.GetFileName(Archivo.FileName))`.
- **El archivo se guarda pero la imagen no se ve.** Se guardó fuera de `wwwroot`. Si está adentro, la dirección es `/imagenes/nombre.jpg`; si está afuera, hace falta configurar los archivos estáticos.
- **El sitio se rompe con un archivo grande.** La petición superó el tope de Kestrel (30 MB) o el del formulario multipart (128 MB), y la excepción ocurre **antes** del código. Se arregla subiendo el límite o bajando el del usuario.
- **Un nombre de archivo "raro" borra o pisa cosas.** El nombre viene del cliente y puede traer una ruta adentro: `Path.GetFileName` antes de armar cualquier ruta.
- **El nombre duplicado pisa el archivo anterior.** Falta el `System.IO.File.Exists` antes de grabar, que es justamente el punto 4 del apunte.
- **La foto del auto no aparece en el listado.** En la tabla quedó el nombre pero se guardó con otro (por ejemplo, con la ruta completa). Hay que grabar en la columna `foto` el mismo nombre que se usó para el archivo.

Ejemplos resueltos

Nota sobre la organización: seguimos en el **mismo proyecto** de las guías anteriores (`LP3-2026-2`) y la **misma base** `administracion` . Los paquetes de EF Core y la cadena de conexión ya están configurados desde la guía 04: **no hay nada que instalar ni que tocar en** `Program.cs` . Las páginas se crean con clic derecho sobre `Pages` → **Agregar** → **Nueva página Razor**, igual que todas las anteriores, y la carpeta `wwwroot/imagenes` se crea con clic derecho sobre `wwwroot` → Agregar → **Nueva carpeta**. Como siempre, los miembros se pegan **adentro** de la clase que genera Visual Studio, no se reemplaza el archivo entero.

Ejemplo 1 — FileUpload + SaveAs: subir un archivo y guardarlo

Es el primero de los cuatro webforms del apunte: uno con un `FileUpload` que graba el archivo en la carpeta del sitio y muestra "Archivo subido". Se crea la página `subir` .

1. **Crear la carpeta** `wwwroot/imagenes` (clic derecho sobre `wwwroot` → Agregar → Nueva carpeta).
2. **Crear la página** `subir` (clic derecho sobre `Pages` → Agregar → Nueva página Razor).
3. **Reemplazar el PageModel** por el del primer bloque: el `IWebHostEnvironment` inyectado, la propiedad `IFormFile` y el `OnPostAsync` que copia el archivo.
4. **Reemplazar la vista** por la del segundo bloque: el formulario con `enctype="multipart/form-data"` , el campo de archivos y el mensaje.
5. **Probar:** entrar a `https://localhost:xxxx/subir` , elegir una imagen y presionar *Confirmar*. Tiene que aparecer "Archivo subido" y el archivo, en la carpeta `wwwroot/imagenes` del proyecto.

PageModel `Pages/subir.cshtml.cs` :

```
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

public class subirModel : PageModel
{
    // El Server del apunte: la clase que sabe dónde vive el sitio en el disco
    private readonly IWebHostEnvironment _env;

    public subirModel(IWebHostEnvironment env)
    {
        _env = env;
    }

    // El control FileUpload del apunte, ahora como propiedad del formulario
    [BindProperty]
    public IFormFile? Archivo { get; set; }

    // La Label1 del apunte
    public string? Mensaje { get; set; }

    public void OnGet()
    {
    }

    // El Button1_Click del apunte (async porque se escribe en el disco)
    public async Task<IActionResult> OnPostAsync()
    {
        if (Archivo is null || Archivo.Length <= 0)
        {
            ModelState.AddModelError(nameof(Archivo), "Debe elegir un archivo");
            return Page();
        }

        // El SaveAs(Server.MapPath(".") + "/" + FileName) del apunte
        string ruta = Path.Combine(_env.WebRootPath, Path.GetFileName(Archivo.FileName));

        using (FileStream destino = new FileStream(ruta, FileMode.Create))
        {
            await Archivo.CopyToAsync(destino);
        }

        Mensaje = "Archivo subido";
        return Page();
    }
}
```

Vista `Pages/subir.cshtml`:

```
@page
@model subirModel
@{
    ViewData["Title"] = "Subir archivo";
}

<h1>Subir archivo</h1>

<!-- enctype="multipart/form-data" es obligatorio: sin esto el archivo no viaja -->
<form method="post" enctype="multipart/form-data">
    <p>
        <label asp-for="Archivo">Archivo:</label>
        <input asp-for="Archivo" type="file" />
        <span asp-validation-for="Archivo" style="color:#b71c1c"></span>
    </p>
    <p>
        <button type="submit">Confirmar</button>
    </p>
</form>

@if (Model.Mensaje is not null)
{
    <p><strong>@Model.Mensaje</strong></p>
}

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

Qué se ve en pantalla: el formulario con el botón "Seleccionar archivo" y el botón *Confirmar*. Sin elegir nada, el mensaje *"Debe elegir un archivo"* aparece al lado del campo. Al elegir una imagen y confirmar, la página vuelve con "Archivo subido" y el archivo queda en `wwwroot/imagenes`, visible desde el *Explorador de soluciones* (botón derecho sobre la carpeta → Actualizar) o desde el disco.

Lectura del código: la propiedad `IFormFile` es el control `FileUpload`; `CopyToAsync` es el `SaveAs`; `_env.WebRootPath` es el `Server.MapPath(".")` y `Path.Combine` reemplaza la concatenación con `"/"`. El chequeo previo (`Archivo is null`) es el caso "no se eligió ningún archivo": el apunte lo dejaba pasar y grababa cualquier cosa, acá se avisa con la misma validación que ya conocemos de la guía 07.

Ejemplo 2 — Guardar el archivo en un subdirectorio

Es el segundo webform del apunte, que cambia el destino por la carpeta `imagenes` y explica cómo crearla. Sobre el mismo código del Ejemplo 1, el único cambio es una palabra en la ruta. Se crea la página `subirimagen`.

1. **Crear la página** `subirimagen`.
2. **Copiar el PageModel** del Ejemplo 1 y cambiar el `Path.Combine`: agregar `"imagenes"` como segmento intermedio.
3. **Copiar la vista** del Ejemplo 1 (solo cambia el `@model`).

4 **Probar:** subir una imagen y comprobar que quedó **adentro** de `wwwroot/imagenes` y no suelta en `wwwroot`.

PageModel `Pages/subirimagen.cshtml.cs` (solo el manejador):

```
// El SaveAs(Server.MapPath(".") + "/imagenes/" + FileName) del apunte:
// el subdirectorio se agrega como un segmento más de la ruta
public async Task<IActionResult> OnPostAsync()
{
    if (Archivo is null || Archivo.Length <= 0)
    {
        ModelState.AddModelError(nameof(Archivo), "Debe elegir un archivo");
        return Page();
    }

    string ruta = Path.Combine(_env.WebRootPath, "imagenes", Path.GetFileName(Archivo.FileName));

    using (FileStream destino = new FileStream(ruta, FileMode.Create))
    {
        await Archivo.CopyToAsync(destino);
    }

    Mensaje = "Archivo subido";
    return Page();
}
```

Qué se ve en pantalla: igual que el Ejemplo 1, pero el archivo aparece en la subcarpeta. Y además —esto es lo nuevo— se puede mostrar desde el navegador con una etiqueta `<img>`: `https://localhost:xxxx/imagenes/nombre.jpg` devuelve la imagen, porque `wwwroot` es la carpeta pública del sitio.

Lectura del código: el apunte creaba la carpeta desde el *Explorador de soluciones* con botón derecho → "Nueva carpeta"; acá es el mismo gesto sobre `wwwroot`. La diferencia entre el Ejemplo 1 y este es **una sola palabra** en `Path.Combine`, que es exactamente la diferencia entre los dos `SaveAs` del apunte. La buena práctica que el apunte justifica —"evitar almacenar los archivos que suben los usuarios en la misma carpeta donde se encuentran las páginas"— se mantiene igual.

Detalle útil: si el proyecto se sube a un repositorio Git, las carpetas vacías no viajan. Una práctica común es dejar un archivo cualquiera (por ejemplo `LEEME.txt`) adentro de `imagenes` para que la carpeta exista en cualquier clon.

Ejemplo 3 — Mostrar las propiedades del archivo subido

Es el tercer webform del apunte: además del `FileUpload` y el botón, muestra el tamaño en bytes, el nombre y el tipo del archivo. Se crea la página `propiedades`.

1 **Crear la página** `propiedades`.

2. **Reemplazar el PageModel**: a las propiedades del Ejemplo 1 se le agregan las tres del archivo (`Tamanio`, `Nombre`, `Tipo`).

3. **Reemplazar la vista**: el mismo formulario, y debajo las tres líneas con los datos.

4. **Probar** con una imagen y con un archivo de otro tipo: el tamaño y el `ContentType` cambian.

PageModel `Pages/propiedades.cshtml.cs`:

```
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

public class propiedadesModel : PageModel
{
    private readonly IWebHostEnvironment _env;

    public propiedadesModel(IWebHostEnvironment env)
    {
        _env = env;
    }

    [BindProperty]
    public IFormFile? Archivo { get; set; }

    public string? Mensaje { get; set; }

    // Las Label2, Label3 y Label4 del apunte
    public string? Tamanio { get; set; }
    public string? Nombre { get; set; }
    public string? Tipo { get; set; }

    public void OnGet()
    {
    }

    public async Task<IActionResult> OnPostAsync()
    {
        if (Archivo is null || Archivo.Length <= 0)
        {
            ModelState.AddModelError(nameof(Archivo), "Debe elegir un archivo");
            return Page();
        }

        string ruta = Path.Combine(_env.WebRootPath, "imagenes", Path.GetFileName(Archivo.FileName));

        using (FileStream destino = new FileStream(ruta, FileMode.Create))
        {
            await Archivo.CopyToAsync(destino);
        }

        Mensaje = "Archivo subido";

        // Las propiedades que el apunte leía en PostedFile, ahora en el propio IFormFile
        Tamanio = Archivo.Length.ToString() + " bytes";
        Nombre = Archivo.FileName;
        Tipo = Archivo.ContentType;

        return Page();
    }
}
```

Vista `Pages/propiedades.cshtml`:

```
@page
@model propiedadesModel
@{
    ViewData["Title"] = "Propiedades del archivo";
}

<h1>Propiedades del archivo subido</h1>

<form method="post" enctype="multipart/form-data">
    <p>
        <label asp-for="Archivo">Archivo:</label>
        <input asp-for="Archivo" type="file" />
        <span asp-validation-for="Archivo" style="color:#b71c1c"></span>
    </p>
    <p>
        <button type="submit">Confirmar</button>
    </p>
</form>

@if (Model.Mensaje is not null)
{
    <p><strong>@Model.Mensaje</strong></p>
    <p>
        Tamaño: @Model.Tamanio<br />
        Nombre: @Model.Nombre<br />
        Tipo: @Model.Tipo
    </p>
}
```

Qué se ve en pantalla: debajo del mensaje, tres líneas con el tamaño en bytes (por ejemplo `184233 bytes`), el nombre con el que se subió el archivo y su tipo (`image/jpeg`, `application/pdf`...). El tamaño es el dato que el apunte señala como la base para **limitar el peso del archivo**.

Lectura del código: `Archivo.Length` es el `PostedFile.ContentLength`, `Archivo.FileName` es el `FileName` y `Archivo.ContentType` es el `PostedFile.ContentType`. El objeto `PostedFile` del apunte no existe como pieza aparte: sus propiedades están en el `IFormFile`, y a `.ToString()` solo hay que llamarlo porque `Length` es un número y el mensaje se arma con textos.

Ejemplo 4 — Validar que no exista otro archivo con el mismo nombre

Es el cuarto webform del apunte: antes de grabar se pregunta si el archivo ya existe; si existe, no se almacena y se avisa. Se crea la página `subirvalidando`.

1. **Crear la página** `subirvalidando`.
2. **Reemplazar el PageModel** por el del bloque: el `if` con `System.IO.File.Exists` y los dos caminos.
3. **Reemplazar la vista** por la del Ejemplo 1 con el `@model` nuevo.
4. **Probar dos veces** con el **mismo archivo**: la primera dice "Archivo subido" y la segunda,

"Existe un archivo con dicho nombre en el servidor".

PageModel `Pages/subirvalidando.cshtml.cs`:

```
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

public class subirvalidandoModel : PageModel
{
    private readonly IWebHostEnvironment _env;

    public subirvalidandoModel(IWebHostEnvironment env)
    {
        _env = env;
    }

    [BindProperty]
    public IFormFile? Archivo { get; set; }

    public string? Mensaje { get; set; }

    public void OnGet()
    {
    }

    public async Task<IActionResult> OnPostAsync()
    {
        if (Archivo is null || Archivo.Length <= 0)
        {
            ModelState.AddModelError(nameof(Archivo), "Debe elegir un archivo");
            return Page();
        }

        string ruta = Path.Combine(_env.WebRootPath, "imagenes", Path.GetFileName(Archivo.FileName));

        // El File.Exists del apunte, con la misma clase System.IO.File
        if (System.IO.File.Exists(ruta))
        {
            Mensaje = "Existe un archivo con dicho nombre en el servidor";
        }
        else
        {
            using (FileStream destino = new FileStream(ruta, FileMode.Create))
            {
                await Archivo.CopyToAsync(destino);
            }

            Mensaje = "Archivo subido";
        }

        return Page();
    }
}
```

Qué se ve en pantalla: con un archivo nuevo, "Archivo subido"; con un nombre que ya está en la carpeta, el aviso y **el archivo anterior intacto**. Es la diferencia con el Ejemplo 1, donde

`FileMode.Create` lo pisaba.

Lectura del código: el método `Exists` del apunte —"retorna true si ya existe un archivo con el nombre que le pasamos como parámetro"— es el mismo método, de la misma clase y con el mismo nombre. Lo único que cambió es cómo se arma la ruta que se le pasa.

Ejemplo 5 — El ejercicio resuelto: las dos tablas y el contexto

Arranca el **ejercicio resuelto** del apunte, el de las tablas `autos` y `marcas`. Antes de las páginas hay que crear las dos tablas y mapearlas como entidades, igual que en la guía 06. Las tablas del apunte son:

```
create table marcas
(
    codigo int identity primary key,      -- #codigo int identidad
    descripcion varchar(30)
);

create table autos
(
    patente char(6) primary key,          -- #patente char(6)
    propietario varchar(50),
    precio float,
    codigomarca int,
    foto varchar(100),                    -- el NOMBRE del archivo, no la imagen
    modelo int
);
```

Se ejecuta ese script en la base `administracion` (con SSMS o desde la ventana de consulta del Explorador de objetos). Como las tablas ya existen, el contexto **solo las mapea**: no hace falta ninguna migración para consultarlas.

1. **Clic derecho sobre la carpeta `Modelos` → Agregar → Clase**, con el nombre `Marca`, y reemplazar el contenido por el de la primera clase.
2. **Repetir con `Auto`**. La columna `precio` es `float`, así que en C# su propiedad es `double`.
3. **Agregar los dos `DbSet` al contexto `AbmContext`**, debajo del de `Usuarios`.
4. **Verificar:** compilar el proyecto (Compilar → Compilar solución). Los nombres de las columnas se revisan al consultar, no al compilar.

Archivo `Modelos/Marca.cs`:

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

// La clase Marca representa la tabla marcas del apunte
[Table("marcas")]
public class Marca
{
    // La columna es int identity primary key: EF Core reconoce la identidad
    // por el tipo y por [Key], no hace falta marcarla a mano
    [Key]
    [Column("codigo")]
    public int Codigo { get; set; }

    // varchar(30) sin not null admite vacío: por eso string?
    [Column("descripcion")]
    public string? Descripcion { get; set; }
}
```

Archivo `Modelos/Auto.cs`:

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

// La clase Auto representa la tabla autos del apunte
[Table("autos")]
public class Auto
{
    // La clave primaria es la patente, y es de texto: se carga a mano
    // (el #patente char(6) del apunte, sin identidad)
    [Key]
    [Column("patente")]
    public string Patente { get; set; } = "";

    [Column("propietario")]
    public string? Propietario { get; set; }

    // La columna es float: en C# le corresponde double
    [Column("precio")]
    public double Precio { get; set; }

    // Acá se guarda el código de la fila de marcas a la que pertenece el auto
    [Column("codigomarca")]
    public int Codigomarca { get; set; }

    // varchar(100): el NOMBRE del archivo de la foto, que está en wwwroot/imagenes
    [Column("foto")]
    public string? Foto { get; set; }

    [Column("modelo")]
    public int Modelo { get; set; }
}
```

Archivo `Modelos/AbmContext.cs` (el contexto de la guía 04, con las dos tablas nuevas):

```
using Microsoft.EntityFrameworkCore;

public class AbmContext : DbContext
{
    public AbmContext(DbContextOptions<AbmContext> options) : base(options)
    {
    }

    // Cada DbSet<> es una tabla vista desde C#
    public DbSet<Usuario> Usuarios => Set<Usuario>();

    // NUEVOS: las dos tablas del ejercicio resuelto
    public DbSet<Marca> Marcas => Set<Marca>();
    public DbSet<Auto> Autos => Set<Auto>();
}
```

Lectura del código: `[Table]` dice a qué tabla corresponde la clase, `[Key]` cuál es la clave primaria —la `patente`, un `char(6)` que en C# es `string` — y `[Column]` a qué columna corresponde cada propiedad. Es el mismo mapeo de la guía 06: en el apunte la tabla se arrastraba desde el Explorador de servidores al formulario, acá cada tabla del ejercicio es una clase y un `DbSet`.

Ejemplo 6 — Punto (a): el ABM de marcas

La parte (a) del apunte pide el mantenimiento de la tabla `marcas` —altas, bajas y modificaciones con un `GridView` —, y para el alta usa el `InsertParameters` del `SqlDataSource`. Se crea la página `abmmarcas`.

1. **Crear la página** `abmmarcas`.
2. **Reemplazar el PageModel** por el del bloque: el contexto inyectado, la carga de la lista, el alta, la modificación y la baja.
3. **Reemplazar la vista** por la tabla con el `@foreach`, el formulario del alta arriba y un formulario por fila para modificar y borrar.
4. **Probar:** dar de alta una marca, modificar la descripción de una fila y borrar otra. Comprobar los cambios en la base con `select * from marcas;`.

PageModel `Pages/abmmarcas.cshtml.cs`:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;

public class abmmarcasModel : PageModel
{
    // El SqlDataSource del apunte: el contexto, inyectado por constructor
    private readonly AbmContext _context;

    public abmmarcasModel(AbmContext context)
    {
        _context = context;
    }

    // Las filas que dibuja el @foreach (el SelectQuery con su GridView)
    public List<Marca> Marcas { get; set; } = new();

    // El TextBox1 del apunte
    [BindProperty]
    public string? Descripcion { get; set; }

    // La Label1 del apunte
    public string? Mensaje { get; set; }

    public void OnGet()
    {
        CargarMarcas();
    }

    // El Button1_Click "Alta" del apunte
    public IActionResult OnPostAlta()
    {
        // El DSMarcas.InsertParameters["descripcion"].DefaultValue = TextBox1.Text
        // más el DSMarcas.Insert() del apunte
        _context.Marcas.Add(new Marca { Descripcion = Descripcion });

        // Add NO toca la base: el insert viaja con SaveChanges
        int cantidad = _context.SaveChanges();

        Mensaje = "La marca fue cargada";

        // El this.TextBox1.Text = "" del apunte
        Descripcion = "";

        if (cantidad <= 0)
        {
            Mensaje = "No se pudo cargar la marca";
        }

        CargarMarcas();
        return Page();
    }

    // La modificación de la fila del GridView (el UpdateQuery del apunte)
    public IActionResult OnPostModificar(int codigo, string? descripcion)
    {
        Marca? marca = _context.Marcas.Find(codigo);

        if (marca is null)
        {
```

```
            Mensaje = "No existe el código";
        }
        else
        {
            marca.Descripcion = descripcion;

            // El update viaja recién acá y SaveChanges devuelve las filas afectadas
            int cantidad = _context.SaveChanges();

            if (cantidad >= 1)
                Mensaje = "Se modificó la marca";
            else
                Mensaje = "No se modificó nada";
        }

        CargarMarcas();
        return Page();
    }

    // La baja de la fila del GridView (el DeleteQuery del apunte)
    public IActionResult OnPostEliminar(int codigo)
    {
        Marca? marca = _context.Marcas.Find(codigo);

        if (marca is null)
        {
            Mensaje = "No existe el código";
        }
        else
        {
            _context.Marcas.Remove(marca);
            _context.SaveChanges();
            Mensaje = "Se borró la marca";
        }

        CargarMarcas();
        return Page();
    }

    private void CargarMarcas()
    {
        // El SelectQuery del apunte, ahora como consulta LINQ
        Marcas = _context.Marcas.OrderBy(m => m.Codigo).ToList();
    }
}
```

Vista `Pages/abmmarcas.cshtml`:

```
@page
@model abmmarcasModel
@{
    ViewData["Title"] = "ABM de marcas";
}

<h1>Marcas</h1>

<!-- El alta: el TextBox1 y el botón "Alta" del apunte -->
<form method="post" asp-page-handler="Alta">
    <p>
        <label asp-for="Descripcion">Descripción:</label>
        <input asp-for="Descripcion" />
        <button type="submit">Alta</button>
    </p>
</form>

@if (Model.Mensaje is not null)
{
    <p><strong>@Model.Mensaje</strong></p>
}

<!-- La grilla: la tabla con el @foreach -->
<table>
    <thead>
        <tr><th>Código</th><th>Descripción</th><th>Acciones</th></tr>
    </thead>
    <tbody>
    @foreach (Marca marca in Model.Marcas)
    {
        <tr>
            <td>@marca.Codigo</td>
            <td>@marca.Descripcion</td>
            <td>
                <form method="post" asp-page-handler="Modificar">
                    <input type="hidden" name="codigo" value="@marca.Codigo" />
                    <input name="descripcion" value="@marca.Descripcion" />
                    <button type="submit">Modificar</button>
                </form>
                <form method="post" asp-page-handler="Eliminar">
                    <input type="hidden" name="codigo" value="@marca.Codigo" />
                    <button type="submit">Borrar</button>
                </form>
            </td>
        </tr>
    }
    </tbody>
</table>
```

Qué se ve en pantalla: una fila por marca, con su código y un campo de texto para la descripción con los botones *Modificar* y *Borrar*; arriba, el alta. Al dar de alta, la fila nueva aparece al recargar y la base queda con el registro.

Lectura del código: las `InsertParameters["descripcion"].DefaultValue` del apunte se convierten en las propiedades del objeto que se crea (`new Marca { Descripcion = Descripcion }`), el `Insert()` en `SaveChanges()` , el `SelectQuery` en la consulta LINQ del `CargarMarcas` , y el `UpdateQuery` / `DeleteQuery` en `Find` más el cambio de la propiedad o `Remove` . Igual que en la guía 06, **cada acción**

es un manejador (`OnPostAlta` , `OnPostModificar` , `OnPostEliminar`) que se elige desde la vista con `asp-page-handler` : eso reemplaza el "Elegir origen de datos" del GridView.

Ejemplo 7 — Punto (b): el alta de autos con la lista de marcas y la foto

La parte (b) del apunte: una página que da de alta un auto, elige la marca en un `DropDownList` y sube la foto con el `FileUpload` , para después llenar los `InsertParameters` del `DSAutos` e insertar. Se crea la página `altaauto` .

1. **Crear la página** `altaauto` .
2. **Reemplazar el PageModel**: el contexto y el `IWebHostEnvironment` inyectados, las propiedades de los campos, la lista de marcas y el manejador que graba la foto y la fila.
3. **Reemplazar la vista**: los campos, el `<select>` de las marcas y el campo de archivo.
4. **Probar:** dar de alta dos o tres autos de marcas distintas, cada uno con su foto, y comprobar con `select * from autos;` que la columna `foto` tiene el **nombre** del archivo.

PageModel `Pages/altaauto.cshtml.cs` :

```
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.AspNetCore.Mvc.Rendering;

public class altaautoModel : PageModel
{
    private readonly AbmContext _context;
    private readonly IWebHostEnvironment _env;

    // El SqlDataSource y el Server del apunte, los dos por constructor
    public altaautoModel(AbmContext context, IWebHostEnvironment env)
    {
        _context = context;
        _env = env;
    }

    // Los TextBox del apunte
    [BindProperty]
    public string? Patente { get; set; }

    [BindProperty]
    public string? Propietario { get; set; }

    [BindProperty]
    public double Precio { get; set; }

    [BindProperty]
    public int Modelo { get; set; }

    // El DropDownList1 del apunte, vinculado al DSMarcas
    [BindProperty]
    public int Codigomarca { get; set; }

    // El FileUpload1 del apunte
    [BindProperty]
    public IFormFile? Foto { get; set; }

    // El origen de datos de la lista de marcas
    public SelectList? MarcasSelect { get; set; }

    // La Label1 del apunte
    public string? Mensaje { get; set; }

    public void OnGet()
    {
        CargarMarcas();
    }

    public async Task<IActionResult> OnPostAsync()
    {
        if (Foto is null || Foto.Length <= 0)
        {
            ModelState.AddModelError(nameof(Foto), "Debe elegir la foto del auto");
        }

        if (!ModelState.IsValid)
        {
            CargarMarcas();
```

```
            return Page();
        }

        // El FileUpload1.SaveAs(Server.MapPath(".") + "/" + FileName) del apunte
        string nombreFoto = Path.GetFileName(Foto!.FileName);
        string ruta = Path.Combine(_env.WebRootPath, "imagenes", nombreFoto);

        using (FileStream destino = new FileStream(ruta, FileMode.Create))
        {
            await Foto.CopyToAsync(destino);
        }

        // Los InsertParameters del DSAutos, ahora como un objeto nuevo
        Auto auto = new Auto
        {
            Patente = Patente!,
            Propietario = Propietario,
            Precio = Precio,
            Modelo = Modelo,
            Codigomarca = Codigomarca,
            // En la tabla se guarda el NOMBRE del archivo, no la imagen
            Foto = nombreFoto
        };

        // El DSAutos.Insert() del apunte
        _context.Autos.Add(auto);
        _context.SaveChanges();

        Mensaje = "Los datos fueron cargados";

        // El apunte limpiaba los TextBox después del alta
        Patente = "";
        Propietario = "";
        Precio = 0;
        Modelo = 0;

        CargarMarcas();
        return Page();
    }

    private void CargarMarcas()
    {
        MarcasSelect = new SelectList(
            _context.Marcas.OrderBy(m => m.Descripcion).ToList(), "Codigo", "Descripcion");
    }
}
```

Vista `Pages/altaauto.cshtml`:

```
@page
@model altaautoModel
@{
    ViewData["Title"] = "Alta de autos";
}

<h1>Alta de autos</h1>

<!-- multipart/form-data: por acá viaja el archivo de la foto -->
<form method="post" enctype="multipart/form-data">
    <p>
        <label asp-for="Patente">Patente:</label>
        <input asp-for="Patente" />
    </p>
    <p>
        <label asp-for="Propietario">Propietario:</label>
        <input asp-for="Propietario" />
    </p>
    <p>
        <label asp-for="Precio">Precio:</label>
        <input asp-for="Precio" />
    </p>
    <p>
        <label asp-for="Modelo">Modelo:</label>
        <input asp-for="Modelo" />
    </p>
    <p>
        <label asp-for="Codigomarca">Marca:</label>
        <select asp-for="Codigomarca" asp-items="Model.MarcasSelect">
            <option value="">-- Seleccione una marca --</option>
        </select>
    </p>
    <p>
        <label asp-for="Foto">Foto:</label>
        <input asp-for="Foto" type="file" />
        <span asp-validation-for="Foto" style="color:#b71c1c"></span>
    </p>
    <p>
        <button type="submit">Confirmar</button>
    </p>
</form>

@if (Model.Mensaje is not null)
{
    <p><strong>@Model.Mensaje</strong></p>
}

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

Qué se ve en pantalla: los cuatro campos, la lista de marcas poblada desde la tabla y el campo de archivo. Sin elegir foto, aparece el mensaje y no se graba nada; con la foto, la página vuelve con "Los datos fueron cargados", los campos de texto vacíos y la fila nueva en la tabla `autos`.

Lectura del código: el `DropDownList1` vinculado al `DSMarcas` es el `<select asp-for="Codigomarca" asp-items="Model.MarcasSelect">` —el `SelectList` es la lista de opciones, con el código como valor y la descripción como texto—. Los seis `InsertParameters["x"].DefaultValue` del apunte son las seis

propiedades del objeto `Auto`, y el `DSAutos.Insert()` es el `Add` más el `SaveChanges`. Fijate en un detalle importante: **lo que se guarda en la columna `foto` es el nombre del archivo** (`nombreFoto`), el mismo que se usó para el `FileStream`; si se guardaran cosas distintas, la foto nunca se encontraría. La limpieza de los `TextBox` del apunte se hace también: el formulario vuelve vacío.

Ejemplo 8 — Punto (c): consulta de un auto por patente

La parte (c) del apunte: se ingresa la patente y se muestran todos los datos del auto, incluida la foto, en una `Label` y un control `Image`. El apunte configura el `SelectQuery` del `DSAutos`, lo pasa a modo `DataReader` y lee una fila con un `SqlDataReader`. Se crea la página `consultaauto`.

1. **Crear la página** `consultaauto`.
2. **Reemplazar el PageModel** por el del bloque: el `Find` por la patente y la marca traída por su clave.
3. **Reemplazar la vista**: el campo de la patente, la `<img>` y la lista de datos.
4. **Probar** con una patente existente y con una que no esté: en el primer caso se ven los datos y la foto; en el segundo, *"No existe un auto con dicha patente"*.

PageModel `Pages/consultaauto.cshtml.cs`:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

public class consultaautoModel : PageModel
{
    private readonly AbmContext _context;

    public consultaautoModel(AbmContext context)
    {
        _context = context;
    }

    // El TextBox1 del apunte
    [BindProperty]
    public string? Patente { get; set; }

    // La fila que se encontró (el registro del SqlDataReader)
    public Auto? Auto { get; set; }

    // La descripción que el apunte traía con el join del SelectQuery
    public string? MarcaDescripcion { get; set; }

    // La Label1 del apunte
    public string? Mensaje { get; set; }

    public void OnGet()
    {
    }

    public IActionResult OnPost()
    {
        if (Patente is null)
        {
            Mensaje = "Ingrese una patente";
            return Page();
        }

        // El DSAutos.SelectParameters["patente"].DefaultValue = TextBox1.Text
        // más el select ... where patente=@patente del apunte: la patente es la
        // clave primaria, así que el Find la busca directo
        Auto = _context.Autos.Find(Patente);

        // El if (registro.Read()) / else del apunte
        if (Auto is null)
        {
            Mensaje = "No existe un auto con dicha patente";
            return Page();
        }

        // La marca que el apunte traía en el mismo select, buscada por su clave
        Marca? marca = _context.Marcas.Find(Auto.Codigomarca);
        MarcaDescripcion = marca?.Descripcion;

        return Page();
    }
}
```

Vista `Pages/consultaauto.cshtml`:

```
@page
@model consultaautoModel
@{
    ViewData["Title"] = "Consulta de un auto";
}

<h1>Consulta de un auto</h1>

<form method="post">
    <p>
        <label asp-for="Patente">Patente:</label>
        <input asp-for="Patente" />
        <button type="submit">Consultar</button>
    </p>
</form>

@if (Model.Auto is not null)
{
    <p>
        <!-- El Image1.ImageUrl = registro["foto"] del apunte -->
        <img src="/imagenes/@Model.Auto.Foto" alt="Foto del auto @Model.Auto.Patente" width="300" />
    </p>
    <p>
        Propietario: @Model.Auto.Propietario<br />
        Precio: @Model.Auto.Precio<br />
        Modelo: @Model.Auto.Modelo<br />
        Marca: @Model.MarcaDescripcion
    </p>
}
else if (Model.Mensaje is not null)
{
    <p><strong>@Model.Mensaje</strong></p>
}
```

Qué se ve en pantalla: al ingresar una patente que existe, la foto y debajo la lista de datos —propietario, precio, modelo y marca—; el apunte mostraba lo mismo en la `Label1` con los saltos de línea armados a mano (`"<br>"`), acá los saltos los pone la vista con `<br />` y los datos salen de las propiedades del objeto.

Lectura del código: el `DataSourceMode = DataReader` y el `SqlDataReader` con su `Read()` del apunte son, acá, `Find` , que trae esa fila y devuelve el objeto tipado (o `null` si no está). El `registro["propietario"]` es `Auto.Propietario` . Y el join del `SelectQuery` —que traía también la `descripcion` de la marca— se resuelve con un segundo `Find` por la clave de la marca: la misma cuenta, con el dato ya tipado.

Ejemplo 9 — Punto (d): todas las fotos de una marca

La parte (d) del apunte: se elige una marca de un `DropDownList` y se muestran **todas** las fotos de los autos de esa marca. La consulta es `select foto from autos where codigomarca=@codigomarca` , y el apunte arma las etiquetas `<img>` como texto dentro de la `Label1` , porque pueden ser varias. Se crea la página `fotosmarca` .

1. **Crear la página** `fotosmarca`.
2. **Reemplazar el PageModel**: la lista de marcas para el `<select>` y la consulta de las fotos por marca.
3. **Reemplazar la vista**: el `<select>`, el botón y el `@foreach` que dibuja una `<img>` por foto.
4. **Probar** con una marca que tenga varios autos: tienen que aparecer todas las fotos, una al lado de la otra.

PageModel `Pages/fotosmarca.cshtml.cs`:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.AspNetCore.Mvc.Rendering;
using Microsoft.EntityFrameworkCore;

public class fotosmarcaModel : PageModel
{
    private readonly AbmContext _context;

    public fotosmarcaModel(AbmContext context)
    {
        _context = context;
    }

    // El DropDownList1 del apunte (el código de la marca elegida)
    [BindProperty]
    public int Codigomarca { get; set; }

    public SelectList? MarcasSelect { get; set; }

    // Las fotos de esa marca: la lista que dibuja la vista
    public List<string?> Fotos { get; set; } = new();

    public string? MarcaDescripcion { get; set; }
    public string? Mensaje { get; set; }

    public void OnGet()
    {
        CargarMarcas();
    }

    // El Button1_Click "Mostrar" del apunte
    public IActionResult OnPost()
    {
        CargarMarcas();

        Marca? marca = _context.Marcas.Find(Codigomarca);
        MarcaDescripcion = marca?.Descripcion;

        // El DSAutos.SelectParameters["codigomarca"].DefaultValue = DropDownList1.SelectedValue
        // más el select foto from autos where codigomarca=@codigomarca del apunte.
        // La igualdad se escribe con Equals(...) por la misma razón que los otros
        // operadores: la lógica es la misma y la consulta SQL generada es la misma.
        Fotos = _context.Autos
            .Where(a => a.Codigomarca.Equals(Codigomarca))
            .Select(a => a.Foto)
            .ToList();

        if (Fotos.Count <= 0)
        {
            Mensaje = "No hay autos cargados de esa marca";
        }

        return Page();
    }

    private void CargarMarcas()
    {
        MarcasSelect = new SelectList(
            _context.Marcas.OrderBy(m => m.Descripcion).ToList(), "Codigo", "Descripcion");
```

```
    }
}
```

Vista `Pages/fotosmarca.cshtml`:

```
@page
@model fotosmarcaModel
@{
    ViewData["Title"] = "Fotos por marca";
}

<h1>Fotos de los autos de una marca</h1>

<form method="post">
    <p>
        <label asp-for="Codigomarca">Marca:</label>
        <select asp-for="Codigomarca" asp-items="Model.MarcasSelect">
            <option value="">-- Seleccione una marca --</option>
        </select>
        <button type="submit">Mostrar</button>
    </p>
</form>

@if (Model.MarcaDescripcion is not null)
{
    <h2>@Model.MarcaDescripcion</h2>
}

@if (Model.Mensaje is not null)
{
    <p><strong>@Model.Mensaje</strong></p>
}

<!-- Una <img> por foto: la vista recorre la lista -->
@foreach (string? foto in Model.Fotos)
{
    <img src="/imagenes/@foto" alt="@foto" width="200" />
}
```

Qué se ve en pantalla: se elige la marca y aparecen **todas** las fotos de esa marca, cada una con su etiqueta `<img>`. Con una marca sin autos, aparece el aviso.

Lectura del código: el apunte explicaba *por qué* no podía usar un control `Image` —porque puede haber varios autos de la misma marca— y por eso armaba las etiquetas como texto dentro de la `Label`. Acá el problema no existe: la consulta devuelve una **lista** de nombres (`Select(a => a.Foto)` es el `select foto` del apunte) y la vista la recorre con un `@foreach`. Es el mismo resultado y, además, sin armar HTML a mano.

Ejemplo 10 — Punto (e): el borrado de un auto por su patente

La parte (e) del apunte: se ingresa la patente y se borra el auto. El apunte consulta primero la fila para recuperar **el nombre del archivo de la foto** y después ejecuta el `DeleteQuery` y el `File.Delete` del archivo. Se crea la página `borrarauto`.

1. **Crear la página** `borrarauto`.
2. **Reemplazar el PageModel**: el `Find` por la patente, el borrado del archivo y el `Remove` más `SaveChanges`.
3. **Reemplazar la vista**: el campo de la patente, el botón y el mensaje.
4. **Probar** con una patente que exista y con otra que no. Después comprobar en la carpeta `imagenes` que **el archivo de la foto también desapareció**.

PageModel `Pages/borrarauto.cshtml.cs`:

```
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

public class borrarautoModel : PageModel
{
    private readonly AbmContext _context;
    private readonly IWebHostEnvironment _env;

    public borrarautoModel(AbmContext context, IWebHostEnvironment env)
    {
        _context = context;
        _env = env;
    }

    // El TextBox1 del apunte
    [BindProperty]
    public string? Patente { get; set; }

    // La Label1 del apunte
    public string? Mensaje { get; set; }

    public void OnGet()
    {
    }

    public IActionResult OnPost()
    {
        if (Patente is null)
        {
            Mensaje = "Ingrese una patente";
            return Page();
        }

        // El SelectQuery del apunte: primero se recupera la fila, para tener
        // el nombre del archivo de la foto (select foto from autos where patente=...)
        Auto? auto = _context.Autos.Find(Patente);

        // El aviso del apunte cuando la cantidad devuelta por el Delete era cero
        if (auto is null)
        {
            Mensaje = "No existe un auto con el valor de patente ingresado";
            return Page();
        }

        // El File.Delete(Server.MapPath(".") + "/" + registro["foto"]) del apunte
        if (auto.Foto is not null)
        {
            string ruta = Path.Combine(_env.WebRootPath, "imagenes", auto.Foto);

            if (System.IO.File.Exists(ruta))
            {
                System.IO.File.Delete(ruta);
            }
        }

        // El DeleteQuery más el Delete() del apunte
        _context.Autos.Remove(auto);
        int cantidad = _context.SaveChanges();
```

```
        if (cantidad >= 1)
        {
            Mensaje = "Se borro el auto con dicha patente";
        }

        return Page();
    }
}
```

Vista `Pages/borrarauto.cshtml`:

```
@page
@model borrarautoModel
@{
    ViewData["Title"] = "Borrado de un auto";
}

<h1>Borrado de un auto</h1>

<form method="post">
    <p>
        <label asp-for="Patente">Patente:</label>
        <input asp-for="Patente" />
        <button type="submit">Borrar</button>
    </p>
</form>

@if (Model.Mensaje is not null)
{
    <p><strong>@Model.Mensaje</strong></p>
}
```

Qué se ve en pantalla: con una patente que existe, el mensaje "Se borro el auto con dicha patente" y —esto es lo importante— **la foto ya no está en la carpeta**. Con una patente que no existe, el aviso del apunte y ningún cambio.

Lectura del código: el apunte hacía los dos comandos en orden: primero el `select` para saber el nombre del archivo, después el `delete` de la fila y el `File.Delete` del archivo. Acá el orden es el mismo: `Find` para tener el nombre de la foto, `File.Delete` para el archivo y `Remove` más `SaveChanges` para la fila. La consulta previa no es un capricho: **una vez borrada la fila ya no hay forma de saber qué archivo correspondía**, y el archivo quedaría huérfano en el disco.

Ejemplo 11 — Punto (f): las fotos de un rango de modelos

La parte (f) del apunte: se ingresan dos valores (un rango de años o modelos) y se muestran todas las fotos de los autos comprendidos en ese rango. La consulta es `select foto from autos where modelo>=@modelo1 and modelo<=@modelo2`. Se crea la página `rangoautos`.

1. **Crear la página** `rangoautos`.
2. **Reemplazar el PageModel**: las dos propiedades del rango y la consulta con los dos límites.

3 **Reemplazar la vista**: los dos campos, el botón y el `@foreach` de las fotos.

4 **Probar** con un rango que incluya varios autos y con un rango vacío. Probar además los **bordes** del rango: los valores ingresados entran (los límites son inclusivos, como en la consulta del apunte).

PageModel `Pages/rangoautos.cshtml.cs`:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;

public class rangoautosModel : PageModel
{
    private readonly AbmContext _context;

    public rangoautosModel(AbmContext context)
    {
        _context = context;
    }

    // Los dos TextBox del apunte
    [BindProperty]
    public int Modelo1 { get; set; }

    [BindProperty]
    public int Modelo2 { get; set; }

    // Las filas que dibuja el @foreach
    public List<Auto> Autos { get; set; } = new();

    public string? Mensaje { get; set; }

    public void OnGet()
    {
    }

    public IActionResult OnPost()
    {
        // El DSAutos.SelectParameters["modelo1"] y ["modelo2"] del apunte
        // más el select foto from autos
        //        where modelo>=@modelo1 and modelo<=@modelo2
        Autos = _context.Autos
            .Where(a => a.Modelo >= Modelo1 && a.Modelo <= Modelo2)
            .OrderBy(a => a.Modelo)
            .ToList();

        if (Autos.Count <= 0)
        {
            Mensaje = "No hay autos en ese rango de modelos";
        }

        return Page();
    }
}
```

Vista `Pages/rangoautos.cshtml`:

```
@page
@model rangoautosModel
@{
    ViewData["Title"] = "Fotos por rango de modelos";
}

<h1>Fotos de autos por rango de modelos</h1>

<form method="post">
    <p>
        <label asp-for="Modelo1">Modelo desde:</label>
        <input asp-for="Modelo1" />
        <label asp-for="Modelo2">hasta:</label>
        <input asp-for="Modelo2" />
        <button type="submit">Mostrar</button>
    </p>
</form>

@if (Model.Mensaje is not null)
{
    <p><strong>@Model.Mensaje</strong></p>
}

@foreach (Auto auto in Model.Autos)
{
    <img src="/imagenes/@auto.Foto" alt="@auto.Patente" width="200" />
}
```

Qué se ve en pantalla: con el rango 2010 a 2020, las fotos de todos los autos cuyo modelo cae adentro —incluidos los de 2010 y 2020—. Con un rango sin autos, el aviso.

Lectura del código: los dos `SelectParameters` del apunte son las dos propiedades `Modelo1` y `Modelo2`, y las dos condiciones del `where` se escriben igual, con `>=` y `<=`: es el único lugar del apunte donde los operadores de la consulta ya venían con esos signos, y en C# funcionan exactamente igual. El `OrderBy` no estaba en el apunte y se agrega para que las fotos salgan de menor a mayor modelo y la prueba sea siempre igual.

Comprobación final (los pasos del apunte)

El apunte cierra cada página ejecutándola y comprobando el resultado. Esa es también la comprobación de esta clase, en el orden del apunte:

1. **Ejemplo 1 y 2:** subir una imagen en `/subir` y en `/subirimagen`. En el primero el archivo queda en `wwwroot`; en el segundo, en `wwwroot/imagenes`, y se ve desde el navegador como `/imagenes/nombre.jpg`.
2. **Ejemplo 3:** subir un archivo en `/propiedades` y comprobar que las tres líneas muestran el tamaño en bytes, el nombre y el tipo.
3. **Ejemplo 4:** subir el mismo archivo dos veces en `/subirvalidando`. La segunda tiene que avisar que ya existe y no grabar.

4 **Ejemplo 6:** abrir `/abmmarcas`, dar de alta una marca, modificar una descripción y borrar una fila. Verificar en la base con `select * from marcas;`.

5 **Ejemplo 7:** abrir `/altaauto` y cargar tres autos de marcas distintas, cada uno con su foto. Verificar con `select patente, codigomarca, foto from autos;` que en `foto` quedó el nombre del archivo.

6 **Ejemplo 8:** consultar una patente existente en `/consultaauto` (tienen que verse los datos y la foto) y una que no exista (tiene que avisar).

7 **Ejemplo 9:** en `/fotosmarca`, elegir una marca con varios autos y comprobar que aparecen **todas** las fotos.

8 **Ejemplo 10:** borrar un auto en `/borrarauto` y comprobar que la fila desapareció de la tabla **y** el archivo de la foto de la carpeta.

9 **Ejemplo 11:** en `/rangoautos`, ingresar un rango que incluya algunos modelos y comprobar que aparecen solo esos, y que los valores de los bordes entran en el resultado.

Con esto está el objetivo del apunte, completo: los cuatro webforms de `FileUpload` (subida simple, subdirectorio, propiedades y validación del nombre duplicado) y el ejercicio resuelto con sus seis puntos (a) a (f), con la foto guardada en el disco y su nombre en la tabla.

✍️ Ejercicios propuestos

Los dos primeros son el **ejercicio propuesto del apunte**, con sus tablas y sus dos puntos; el tercero es un desafío que usa lo que ya se resolvió en el ejercicio resuelto (el borrado con `File.Delete`). Se resuelven en el **mismo proyecto**, creando las tablas con el script y una página por punto.

> **Ejercicio 1 — El apunte, punto (a): alta de libros y temas con la foto de la tapa**
>
> Las tablas del apunte son `libros (codigo, titulo, descripcion, fototapa, codigotema)` y `temas (codigo, descripcion)`. Como la tabla `libros` no declara clave, se la declara igual (`codigo int identity primary key`) para poder dar de baja y modificar de forma confiable.
>
> La consigna del apunte: *"Confeccionar el alta de las tablas libros y temas (permitir seleccionar el tema mediante una lista). Hacer el Upload de la imagen al servidor. Validar que se ingresen obligatoriamente el título, descripción y selección de un archivo."*
>
> Pedí: la entidad `Libro` y la entidad `Tema` con sus atributos de mapeo, los dos `DbSet` en el contexto, la página del alta de temas (con el alta simple de la guía 06) y la página del alta de libros con el `<select>` de temas, el campo de archivo y las tres validaciones obligatorias (`[Required]` en el título y la descripción, y `Archivo is null` para el archivo).
>
> En la columna `fototapa` tiene que quedar el **nombre** del archivo guardado en `wwwroot/imagenes`.

Ejercicio 2 — El apunte, punto (b): mostrar la tapa a partir del código

La consigna del apunte: *"Ingresar por teclado el código de un libro y luego mostrar la foto de la tapa."*

Es el mismo camino que el punto (c) del ejercicio resuelto: el campo del código, un `Find` por la clave y una etiqueta `<img>` que apunta a `/imagenes/nombre.jpg`. Además del código, mostrá el título, la descripción y la descripción del tema (que sale con un segundo `Find` por la clave del tema).

Probá con un código que exista y con uno que no: en el primer caso tiene que verse la tapa; en el segundo, el aviso de que no existe ese libro.

Desafío — Baja de un libro con su tapa

Agregá la baja de un libro por su código, con el mismo criterio que el punto (e) del ejercicio resuelto: primero recuperar la fila para saber el nombre del archivo de la tapa, borrar el archivo del disco con `System.IO.File.Delete` y después la fila con `Remove` y `SaveChanges`.

Antes de borrar, mostrá los datos del libro y su tapa para que el usuario confirme; y al terminar, comprobá en la carpeta `wwwroot/imagenes` que el archivo ya no está. Un detalle para pensar y responder en un comentario: **¿qué pasa si dos libros comparten la misma tapa** (dos libros cargados con el mismo nombre de archivo)? ¿El borrado de uno rompe el otro?

📌 Síntesis y cierre

En esta clase tomamos el control `FileUpload` del apunte —arrastrado al WebForm, con su `SaveAs` y su `Server.MapPath` — y lo escribimos con las piezas del stack nuevo: un campo de archivos en el formulario, una propiedad `IFormFile` y la copia del contenido a un `FileStream`.

- **El control es una etiqueta y una propiedad:** `<input asp-for="Archivo" type="file" />` en la vista, `[BindProperty] public IFormFile? Archivo` en el PageModel. Y el `enctype="multipart/form-data"` en el formulario es obligatorio: sin eso el archivo no viaja.
- **El `SaveAs` es una copia:** `Archivo.CopyToAsync(new FileStream(ruta, FileMode.Create))`, dentro de un `using` y con `await`. El `Server.MapPath(".")` es `_env.WebRootPath`, inyectado por constructor.
- **Las propiedades están en el propio `IFormFile`:** `Length` (el `PostedFile.ContentLength`), `FileName` y `ContentType`. El objeto `PostedFile` desaparece.
- **`File.Exists` y `File.Delete` son los mismos métodos:** la clase `System.IO.File` no cambió; solo cambia cómo se arma la ruta que se les pasa.
- **Validar el archivo:** el tipo con `[FileExtensions]`, el peso con `Archivo.Length` y el mensaje al usuario, y el tope de la petición con `[RequestSizeLimit]` y `[RequestFormLimits]`. Siempre `Path.GetFileName` antes de armar la ruta.
- **El archivo en el disco, el nombre en la tabla:** la carpeta `wwwroot/imagenes` guarda las fotos, la columna `foto` guarda el nombre, y la vista muestra la imagen con `<img`

`src="/imagenes/@Model.Auto.Foto" />`. Cuando hay varias, un `@foreach` y una `<img>` por vuelta.

Conceptos clave: `IFormFile`, `enctype="multipart/form-data"`, `CopyToAsync`, `FileStream`, `FileMode.Create`, `IWebHostEnvironment.WebRootPath`, `ContentRootPath`, `Path.Combine`, `Path.GetFileName`, `FileName`, `Length`, `ContentType`, `FileExtensions`, `RequestSizeLimit`, `MultipartBodyLengthLimit`, `System.IO.File.Exists`, `System.IO.File.Delete`, `asp-for`, `asp-items`, `SelectList`, `OnPostAsync`, EF Core (`Add`, `Find`, `Remove`, `SaveChanges`).

Para la próxima clase: traer el sitio con el alta de autos funcionando (la foto en `wwwroot/imagenes` y su nombre en la tabla), la consulta por patente y las dos pantallas de fotos —la de una marca y la del rango de modelos—, probadas con los nueve puntos de la comprobación final.

📚 Bibliografía

- Microsoft Learn. *Upload files in ASP.NET Core (escenarios con `IFormFile`, `enctype="multipart/form-data"`, copiar a un `FileStream`, validación y límites de tamaño).* https://learn.microsoft.com/aspnet/core/mvc/models/file-uploads
- Microsoft Learn. *IFormFile interface (`FileName`, `Length`, `ContentType`, `OpenReadStream`, `CopyToAsync`).* https://learn.microsoft.com/dotnet/api/microsoft.aspnetcore.http.iformfile
- Microsoft Learn. *Serve static files in ASP.NET Core apps (`IWebHostEnvironment.WebRootPath` y la carpeta `wwwroot`).* https://learn.microsoft.com/aspnet/core/fundamentals/static-files
- Microsoft Learn. *FileExtensionsAttribute Class (validar la extensión del archivo con un atributo).* https://learn.microsoft.com/dotnet/api/system.componentmodel.dataannotations.fileextensionsattribute
- Microsoft Learn. *RequestSizeLimitAttribute y RequestFormLimitsAttribute (el tope de la petición y del formulario multipart).* https://learn.microsoft.com/dotnet/api/microsoft.aspnetcore.mvc.requestsizelimitattribute
- Microsoft Learn. *Tag Helpers in forms — The Input Tag Helper (por qué `type="file"` se escribe explícito).* https://learn.microsoft.com/aspnet/core/mvc/views/working-with-forms
- Microsoft Learn. *File Class (`Exists` y `Delete`, los métodos del apunte).* https://learn.microsoft.com/dotnet/api/system.io.file
- Microsoft Learn. *FileStream Class y Path.Combine / Path.GetFileName.* https://learn.microsoft.com/dotnet/api/system.io.filestream
- Microsoft Learn. *Razor Pages architecture and concepts (handlers `OnGet` / `OnPostAsync` y `asp-page-handler`).* https://learn.microsoft.com/aspnet/core/razor-pages/
- Microsoft Learn. *Model binding in ASP.NET Core (cómo llega el archivo del formulario a la propiedad del PageModel).* https://learn.microsoft.com/aspnet/core/mvc/models/model-binding
- Microsoft Learn. *DbContext lifetime, configuration and initialization (el `AbmContext` de la guía 04).* https://learn.microsoft.com/ef/core/dbcontext-configuration/

archivos)